Parallel programming with OpenMP

Université Lorraine - 2025 - 2026

General presentation

The OpenMP library is included in most C/C++ compilers (gcc,...). Let's remind that OpenMP works through *directives* given to the *pre-processor*. To take these directives into account, a specific compiler option must be used. The name of this option may vary from one compiler to another. So, it is recommended to refer to the compiler's documentation.

In our case, we use gcc and the option is -fopenmp. So, a typical compilation command is as follows : `gcc -fopenmp source.c -o executable`

In addition, it is recommended to use the option producing warnings (-Wall) on code inconsistencies as well as the mathematical library (-lm).

An archive is available on the Arche website that contains the C source files and a CMakeLists.txt.

This file allows the use of cmake that provides *automatic* configuration and compilation. To compile, you just need to go in the directory of the source files and type the commands on the right.

```
mkdir build
cd build
cmake ..
make
```

Exercise 1 : First experiments

When compiling with OpenMP, the constant _OPENMP is defined that allows to detect if OpenMP can be used in the program. This is useful for example to perform a conditional inclusion of the OpenMP API. This can be seen at the beginning of the source files.

In this first exercise, you have to edit the file ex_parallel.c. This program creates a parallel region and displays a few information.

a) Compile and execute the program. What do you observe ?

b) In the file CMakeLists.txt, replace YES by NO in the line `set(OMP "YES")` and execute the cmake and make commands in order to recompile the programs. What do you observe during the compilation ? And what about the execution of the ex_parallel program ?

c) Revert the modification in file CMakeLists.txt and recompile. Execute the program with the following command `OMP_NUM_THREADS=12 ./ex_parallel` that specifies the default number of threads to use in the program, thanks to the OMP_NUM_THREADS environment variable. What do you observe ?

d) Add the optional clause `num_threads(3)` after the `parallel` directive in the source file. Recompile and execute *with* and *without* the environment variable.

What do you observe ?

Exercise 2 : A first parallel loop

The program in the source file ex_loop.c executes a loop.

a) Compile and execute this program. What can be observed concerning the iterations of the loop ?

b) Add the clause `schedule(static, 2)` after the OpenMP directive `for`. Recompile and execute the program. Compare with the previous execution.

c) Comment the clause `schedule(static, 2)` as well as the `printf` in the loop, and set the number of iterations to 1000000. Recompile and execute several times the program. What about the validity of the program ?

d) In order to obtain a valid result, add the mutual exclusion directive `#pragma omp atomic` on the line just above the instruction `sum++;`.
Recompile and execute several times. Compare the result and computation time to the previous versions of the program.

e) Comment the directive `atomic` and add the clause `reduction(+:sum)` to the directive `for`. Recompile and execute several times. What can be observed ?

Exercise 3 : An imbalanced loop

The program in the source file `ex_balancing.c` executes an *imbalanced* loop in which the iterations have different execution times.

a) Compile and execute the program. Note the computation times.

b) Add the option `nowait` to the directive `for`. Recompile and execute the program. What do you observe ? How can we explain the execution times of the previous version ?

c) Add the clause `schedule(dynamic)` after the `nowait`. Recompile and execute the program. What do you observe ?

Exercise 4 : Nested parallelism

In the source file `ex_nesting.c`, several nested parallel regions are created and some displays are done at each level.

a) Compile and execute the program. What can you say about the threads of levels 2 and 3 according to what is expected for each region ?

b) Add the instruction `omp_set_nested(1);` before the first parallel region.
Recompile and execute the program. Compare to previous results ?

Exercise 5 : Generation of tasks

The source file `ex_tasks.c` contains a variant of the imbalanced loop seen before.

a) Compile and execute the program. Explain the displays.

b) Place the computation loop in a `single` region, as shown in the following code :

```
#pragma omp single
{
  int i;

  for(...){
    ...
  }
}
```

Recompile and execute the program. What do you observe ?

c) Add a directive `task` just before the call of function `work`. Recompile and execute the program. Explain the results.

Exercise 6 : Application to a more concrete example

The objective of this exercise is to use the notions seen previously to obtain a parallel version of the program contained in the source file `ex_occurrences.c` that counts the number of appearances of each letter of the alphabet in a text.